Destiny Brewington: dkb2140
Divya Shanmugam: das2304

PostgreSQL Account: dkb2140

Changes to Schema:

- *Performer_ID* is now the primary key of the *Performer* table, instead of both *performer_ID* and *performerName*.
- Removed *Singer* and *Band* boolean from under *Performer* table (not supported by SQL).
- Removed *Headliner* and *Opener* boolean from under *Concert* table (not supported by SQL).
- Added *Rating* table, to capture the ratings relationship between *Customer* and *Performer*.
- Added *average_rating* column to *Performer* table.
- Editing *Tickets* table so now each ticket instance is associated with a customer purchase. Now, the table includes the columns *ticket_ID, customer_ID, date_purchase, and time_purchased.*
- Moved *num_tix_avail* and *tix_price* from *Tickets* table to *Concerts* table.
- Removed *average_ratings* from *Venue* table. Felt as though performer rating was more relevant (feedback from Part 1)
- Turned double required participation into required participation by only one of the entities
    - For example, the relationship between Customer and Entity in Part 1 had double required participation. In Part 2, per Kaushal's suggestion, the relationship between *Customer* and *Ticket* only requires participation of *Ticket* due to the Foreign Key constraint that requires a customer (and concert) to be associated with each ticket.

Other Considerations:

- Plan to add delete reviews function to future interface design

Queries:

**Returns the three most popular concert venues (by tickets sold).**

```
SELECT v.venue_name, v.city, v.country,
COUNT(t.ticket_id) AS total_tickets
FROM Tickets t
JOIN Concert c ON t.concert_id = c.concert_id
JOIN Venue v ON c.venue_id = v.venue_id
GROUP BY v.venue_name, v.city, v.country
ORDER BY total_tickets DESC
LIMIT 3;
```

**Returns the customers who have spent the most money.**

```
SELECT c.customer_id, c.firstname, c.lastname,
SUM(x.tix_price) AS total_spent
FROM Customer c
JOIN Tickets t ON c.customer_id = t.customer_id
JOIN Concert x ON t.concert_id = x.concert_id
GROUP BY c.customer_id, c.firstName, c.lastName
ORDER BY total_spent DESC
LIMIT 5;
```

**Returns the performer that sold the most tickets.**

```
SELECT p.performer_id, p.performer_name,
COUNT(t.ticket_id) AS total_tickets_sold
FROM Performer p
JOIN Concert c ON p.performer_id = c.performer_id
JOIN Tickets t ON c.concert_id = t.concert_id
GROUP BY p.performer_id, p.performer_name
ORDER BY total_tickets_sold DESC
LIMIT 1;
```